

[Área personal](#) / [Mis cursos](#) / [Programación III](#) / [SEGUNDO PARCIAL - 9 de Junio](#) / [SEGUNDO PARCIAL JUEVES 09 06 2022](#)

Comenzado el jueves, 9 de junio de 2022, 11:00

Estado Finalizado

Finalizado en jueves, 9 de junio de 2022, 12:02

**Tiempo
empleado** 1 hora 1 minutos

Calificación Sin calificar aún

Pregunta **1**

Correcta

Se puntúa 1,00 sobre 1,00

La serialización es una forma de hacer una clonación

- ☐ a. F
- ☒ b. V ✓

Respuesta correcta

La respuesta correcta es:

V

Pregunta **2**

Finalizado

Puntúa como 5,00

De un ejemplo de un caso donde sea conveniente utilizar el patrón State.

El ejemplo no debe ser ninguno de los vistos en clase ni en el TPE.

Bosqueje la solución haciendo un diagrama de clases completo (o escribiendo de las clases la jerarquía de herencia, los atributos y las cabeceras de métodos).

Establezca los atributos que determinarán los diferentes estados del objeto, determine los estados, los métodos, etc.

Solamente el diagrama de clases con atributos y cabecera de métodos.

No es necesario que haga un gráfico.

Se podría pensar en un termo de mate, que puede estar cargado, vacío, cebando o cargando.

Interface de estados:

```
public interface I_State{  
    public void cargado();  
    public void vacio();  
    public void cebando();  
    public void cargando();  
    public void informa();  
}
```

El termo puede pasar de cargado a cebando, pero no a cargando ni a vacía. De vacío puede pasar solamente a cargando. De cargando únicamente puede pasar al estado de cargado. Y de cebando puede volver a cargado o puede vaciarse.

Aclaración: cargado no significa lleno, sino que aun hay contenido en su interior.

Aclaración 2: Faltan atributos para llevar control del agua en su interior, por cuestiones de tiempo solamente implemente el esqueleto del patrón state.

Clase termo:

```
public class Termo{  
    private I_State estado;  
  
    public Termo(String marca){  
        estado = new vacioState(this);  
        this.marca = marca;  
    }  
  
    protected void setEstado(I_State estado){  
        this.estado = estado;  
    }  
  
    public void cargado(){  
        this.estado.cargando();  
    }  
  
    public void vacio(){  
        this.estado.cargando();  
    }  
}
```

```
public void cebando(){
    this.estado.cargando();
}

public void cargando(){
    this.estado.cargando();
}

public void informa(){
    this.estado.informa();
}
}
```

Clases de los estados:

```
public class vacioState implements I_State{
    Termo termo;

    vacioState(Termo t){
        this.termo = t;
    }

    void vacio(){
        System.out.println("El termo ya se encuentra vacio.");
    }

    void cargado(){
        System.out.println("No puede pasar de vacio a cargado, antes debe llenarse.");
    }

    void cebando(){
        System.out.println("No puede cebar porque esta vacio, antes debe llenarse.");
    }

    void cargando(){
        this.termo.setEstado(new cargandoState(termo));
        System.out.println("Se esta cargando el termo.");
    }

    void informa(){
        System.out.println("El termo se encuentra vacio.");
    }
}
```

```
public class cargadoState implements I_State{
    Termo termo;
```

```
    cargadoState(Termo t){
        this.termo = t;
```

```
}

void vacio(){
    System.out.println("Para que se vacie antes debe usarse.");
}

void cargado(){
    System.out.println("El termo se encuentra cargado.");
}

void cebando(){
    this.termo.setEstado(new cebandoState(termo));
    System.out.println("Se ceba un mate.");
}

void cargando(){
    System.out.println("El termo se encuentra cargado, no hace falta volverlo a cargar.");
}

void informa(){
    System.out.println("El termo esta cargado.");
}
}

public class cebandoState implements I_State{
    Termo termo;

    cebandoState(Termo t){
        this.termo = t;
    }

    void vacio(){
        this.termo.setEstado(new vacioState(termo));
        System.out.println("Se vacio el termo.");
    }

    void cargado(){
        this.termo.setEstado(new cargadoState(termo));
        System.out.println("El termo todavia tiene agua.");
    }

    void cebando(){
        System.out.println("El termo se encuentra en ese estado.");
    }

    void cargando(){
        System.out.println("No puede cargarse un termo mientras se esta cebado un mate.");
    }
}
```

```
void informa(){
    System.out.println("El termo se encuentra cebando un mate.");
}

}

public class cargandoState implements I_State{
    Termo termo;

    cargadoState(Termo t){
        this.termo = t;
    }

    void vacio(){
        System.out.println("El termo no puede estar vacio si se lo esta cargando.");
    }

    void cargado(){
        this.termo.setEstado(new cargadoState(termo));
        System.out.println("Se cargo el termo.");
    }

    void cebando(){
        System.out.println("No puede cebarse un mate mientras se carga el termo.");
    }

    void cargando(){
        System.out.println("El termo ya se encuentra cargandose.");
    }

    void informa(){
        System.out.println("El termo se encuentra cargandose.");
    }

}
```

Pregunta **3**

Correcta

Se puntúa 1,00 sobre 1,00

En el Patrón MVC.

¿De quién es la responsabilidad de validar el formato correcto de los datos que el usuario ingresa?

Seleccione una:

- ☐ a. Modelo
- ☒ b. Vista ✓
- ☐ c. Controlador

Respuesta correcta

La respuesta correcta es: Vista

Pregunta **4**

Incorrecta

Se puntúa -0,50 sobre 1,00

Es correcta la implementación de los siguientes métodos?

Correcta: 1

Incorrecta: -0,5

```
public class Primero
{
    private String nombre;
    private long dni; // identificador que no se repite
    private Date fecha_nac;

    public Primero()
    {
        super();
    }
    @Override
    public boolean equals(Object object)
    {
        if (this == object)
        {
            return true;
        }
        if (!(object instanceof Primero))
        {
            return false;
        }
        final Primero other = (Primero) object;
        if (dni != other.dni)
        {
            return false;
        }
        return true;
    }
    @Override
    public int hashCode()
    {
        final int PRIME = 37;
        int result = 1;
        result = PRIME * result + ((nombre == null) ? 0 : nombre.hashCode());
        result = PRIME * result + (int) (dni ^ (dni >>> 32));
        result = PRIME * result + ((fecha_nac == null) ? 0 : fecha_nac.hashCode());
        return result;
    }
}
```

- ☒ a. V 
- ☐ b. F

Respuesta incorrecta.

La respuesta correcta es:

F

Pregunta **5**

Correcta

Se puntúa 1,00 sobre 1,00

Cuando un hilo ejecuta un método synchronized de un recurso compartido, el resto de los hilos:

Correcta: 1

Incorrecta: -0,5

Seleccione una:

- ☐ a. Pueden acceder a cualquier método de la clase (del recurso) siempre que sea public
- ☐ b. No pueden acceder a este método pero si a cualquiera de los demás métodos de la clase (del recurso)
- ☐ c. No pueden acceder a ningún método de la clase (del recurso).
- ☒ d. No pueden acceder a ningún método synchronized de ese recurso, pero si a cualquier método no synchronized ✓

Respuesta correcta

La respuesta correcta es: No pueden acceder a ningún método synchronized de ese recurso, pero si a cualquier método no synchronized

Pregunta **6**

Correcta

Se puntúa 1,00 sobre 1,00

Elija la/s respuesta/s correctas. Una elección equivocada tiene penalización.
Según la clasificación, el patrón State es:

Correcta vale 1

Incorrecta vale -0,5

Seleccione una:

- ☐ a. Estructural
- ☐ b. Creacional
- ☒ c. de Comportamiento ✓

Respuesta correcta

de Comportamiento

La respuesta correcta es: de Comportamiento

Pregunta **7**

Correcta

Se puntúa 1,00 sobre 1,00

Para poder persistir un objeto utilizando Serealización Binaria

Seleccione una:

- ☒ a. Todos los elementos que queremos persistir deben implementar la interfaz Serializable ✓
- ☐ b. Ninguna de las anteriores
- ☐ c. Tener un constructor público sin parámetros y getters y setters públicos
- ☐ d. Tener un constructor público sin parámetros

Respuesta correcta

La respuesta correcta es: Todos los elementos que queremos persistir deben implementar la interfaz Serializable

Pregunta **8**

Correcta

Se puntúa 1,00 sobre 1,00

Un método estático no puede ser sincronizado ya que el método start() de Thread es de instancia y no estático.

- ☒ a. F ✓
- ☐ b. V

Respuesta correcta

La respuesta correcta es:

F

Pregunta **9**

Correcta

Se puntúa 1,00 sobre 1,00

La función específica del método wait() de un recurso compartido es poner en espera al hilo que lo contiene y liberar al recurso compartido

Correcto: 1

Incorrecto: -1

- ☒ a. V ✓
- ☐ b. F

Respuesta correcta

La respuesta correcta es:

V

Pregunta **10**

Correcta

Se puntúa 1,00 sobre 1,00

Es imposible implementar el Patrón MVC dentro de un programa para desplegar en la WEB (con un browser como vista del usuario)

Correcta: 1

Incorrecta: -0,5

- ☒ a. F ✓
- ☐ b. V

Respuesta correcta

La respuesta correcta es:

F

Pregunta **11**

Correcta

Se puntúa 1,00 sobre 1,00

El patrón Observer/Observable está pensado para aplicaciones WEB.

Correcta: 1

Incorrecta: -0,5

- ☐ a. V
- ☒ b. F ✓

Respuesta correcta

La respuesta correcta es:

F

Pregunta **12**

Correcta

Se puntúa 1,00 sobre 1,00

Supongamos el caso de un objeto de tipo Observable que está siendo observado por varios objetos de tipo Observer a la vez. En ese contexto, el patrón Observer Observable está pensado para:

Correcta: 1

Incorrecta: -1

Seleccione una:

- ☐ a. que un objeto observable notifique a un objeto observer a elección.
- ☒ b. Que un objeto observable notifique a todos los objetos observer a la vez. ✓

Respuesta correcta

El patrón permite que un objeto observable notifique a todos sus observers a la vez

La respuesta correcta es: Que un objeto observable notifique a todos los objetos observer a la vez.

Pregunta **13**

Correcta

Se puntúa 1,00 sobre 1,00

El uso de tipos genéricos permite la comprobación de tipos correctos en tiempo de compilación.

Valor respuesta correcta: 1

Valor respuesta incorrecta: -0,5

- ☐ a. F
- ☒ b. V ✓

Respuesta correcta

La respuesta correcta es:

V

Pregunta **14**

Correcta

Se puntúa 1,00 sobre 1,00

Elija la/s respuesta/s correctas. Una elección equivocada tiene penalización.
El patrón State permite:

Correcta vale 1

Incorrecta vale -0,5

Seleccione una:

- ☐ a. Permite añadir responsabilidades a un objeto en tiempo de ejecución
- ☒ b. Permitir que un objeto cambie el comportamiento al cambiar el estado ✓
- ☐ c. Determinar un algoritmo general en la clase base y determinar los comportamientos particulares a través del mecanismo de herencia

Respuesta correcta

Permitir que un objeto cambie el comportamiento al cambiar el estado

La respuesta correcta es: Permitir que un objeto cambie el comportamiento al cambiar el estado

Pregunta **15**

Finalizado

Puntúa como 5,00

Determine el código del recurso compartido para el siguiente caso:

Se tiene una biblioteca con un catalogo de 20 diferentes libros. De cada libro existen varios ejemplares/copias.

El socio de la biblioteca solicita un determinado libro del catalogo. Si está disponible alguna copia del libro, se lo lleva, si no está disponible, queda en espera hasta que lo devuelvan.

Diseñe y escriba código java del recurso compartido.

Escriba una versión con wait/notify y otra con semáforos. Las dos. La de semáforos vale menos que la de wait/notify.

Clase biblioteca:

```
public class Biblioteca{

    private HashMap<Integer, Libros> libros= new HashMap<Integer, Libros>();

    public void agregaLibro(int nro, String titulo) {
        this.libros.put(nro, new Libros(nro, titulo));
    }

    public synchronized Libros retiraLibro(Socio s, int id) {
        Libros l1;
        System.out.println("El socio "+ s.getNombre()+ " esta tratando de retirar un libro");
        while(!this.libros.containsKey(id)) {
            System.out.println("El libro que desea retirar "+s.getNombre()+ " no se encuentra disponible");
            try {
                wait();
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
        System.out.println("El socio "+ s.getNombre()+ " retiro el libro "+ this.libros.get(id).getTitulo());
        l1=this.libros.get(id);
        this.libros.remove(id);
        return l1;
    }

    public synchronized void devuelveLibro(Socio s,Libros l1) {
        System.out.println("El socio "+ s.getNombre()+ " devuelve el libro "+ l1.getTitulo());
        this.libros.put(l1.getIdLibro(), l1);
        notifyAll();
    }

}

public class Libro{
    private int idLibro;
    private String titulo;
```

```
public Libros(int idLibre, String titulo) {
    this.idLibro = idLibre;
    this.titulo = titulo;
}

public int getIdLibre() {
    return idLibro;
}

public String getTitulo() {
    return titulo;
}

}

public class Socio extends Thread{
    private String nombre;
    private Biblioteca b;
    private int nrolibro;

    public Socio(String nombre, Biblioteca b, int libro) {
        this.nombre = nombre;
        this.b = b;
        this.nrolibro = libro;
    }

    public String getNombre() {
        return nombre;
    }

    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            Libros l1 = b.retiraLibro(this, nrolibro);

            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }

            b.devuelveLibro(this, l1);
        }
    }
}
```

Pregunta **16**

Correcta

Se puntúa 1,00 sobre 1,00

En el patrón MVC, la Vista es la encargada de gestionar los eventos producidos por los componentes activos (botones, enlaces, etc).

Correcta: 1

Incorrecta: -0,5

- ☐ a. V
- ☒ b. F ✓

Respuesta correcta

La respuesta correcta es:

F

Pregunta **17**

Correcta

Se puntúa 1,00 sobre 1,00

La función específica de la instrucción wait() es evitar que dos o más hilos accedan en forma simultánea a un recurso compartido

Correcta: 1

Incorrecta: -1

- ☐ a. V
- ☒ b. F ✓

Respuesta correcta

La respuesta correcta es:

F

Pregunta **18**

Correcta

Se puntúa 1,00 sobre 1,00

Los objetos referenciados por las variables v1 y v2 son de distinto tipo

```
Sea una Clase genérica Clase1<T> y sean:  
Clase1<String> v1 = new Clase1<String>();  
Clase2<Integer>v2 = new Clase2<Integer>();
```

- ☒ a. F ✓
- ☐ b. V

Respuesta correcta

La respuesta correcta es:

F

[← Ejemplo Persistencia cuando hay patron Singleton](#)

Ir a...